VU PARALLEL COMPUTING, 2026S

Multithreading II

OUTLINE

Motivation and Memory Model

Threads

Mutexes

Lock guards, locks

Some remarks on locks

Atomics

Atomics and the memory model

SOME REMINDERS

Compexity of Modern CPUs

- Memory Hierarchy and costs of moving data

Hardware

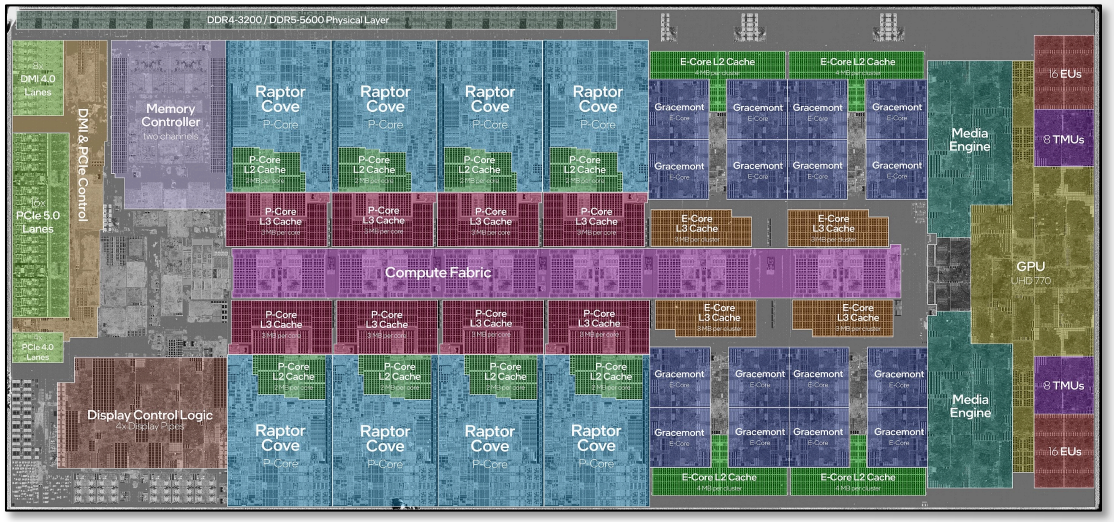
- ILP, speculation, out-of-order execution
- Caches

Compiler

- Optimizations
- Reordering of loads and stores

Optimizations are good

- We don’t want get in the way
- We want to instruct HW/Compilers to do it correcty



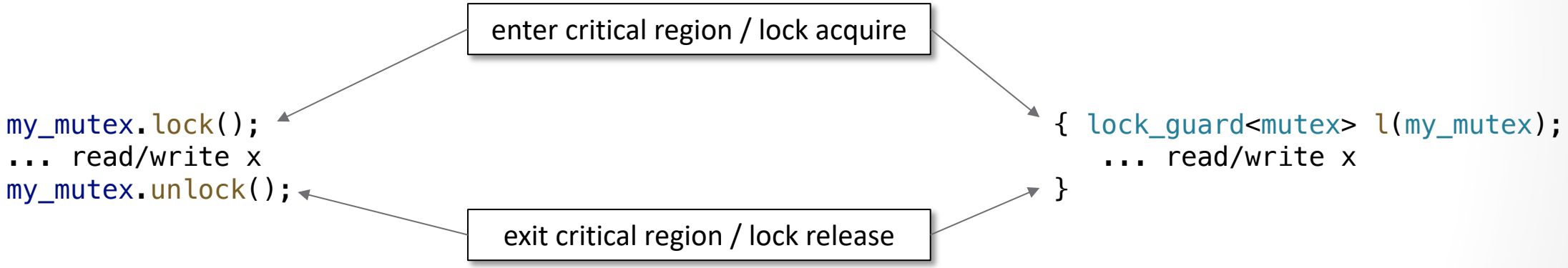
Intel Raptor Lake (Core i9-13900K)
https://en.wikipedia.org/wiki/Raptor_Lake

CRITICAL REGIONS (REMINDER)

A piece of code that mustn't be simultaneously accessed by more than one thread

- Must execute in isolation wrt. other program code

Locks (Mutexes)

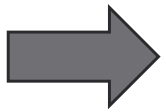


University of Vienna

COMPILER TRANSFORMATIONS (REMINDER)

Remember this one?

```
x = "life";
y = "universe";
z = "everything";
```



```
z = "everything";
y = "universe";
x = "life";
```

What can be reordered now?

```
x = "life";
mut.lock(); (acquire)

y = "universe";


mut.unlock(); (release)
z = "everything";
```

```
mut.lock();

z = "everything";
y = "universe";
x = "life";
mut.unlock();
```

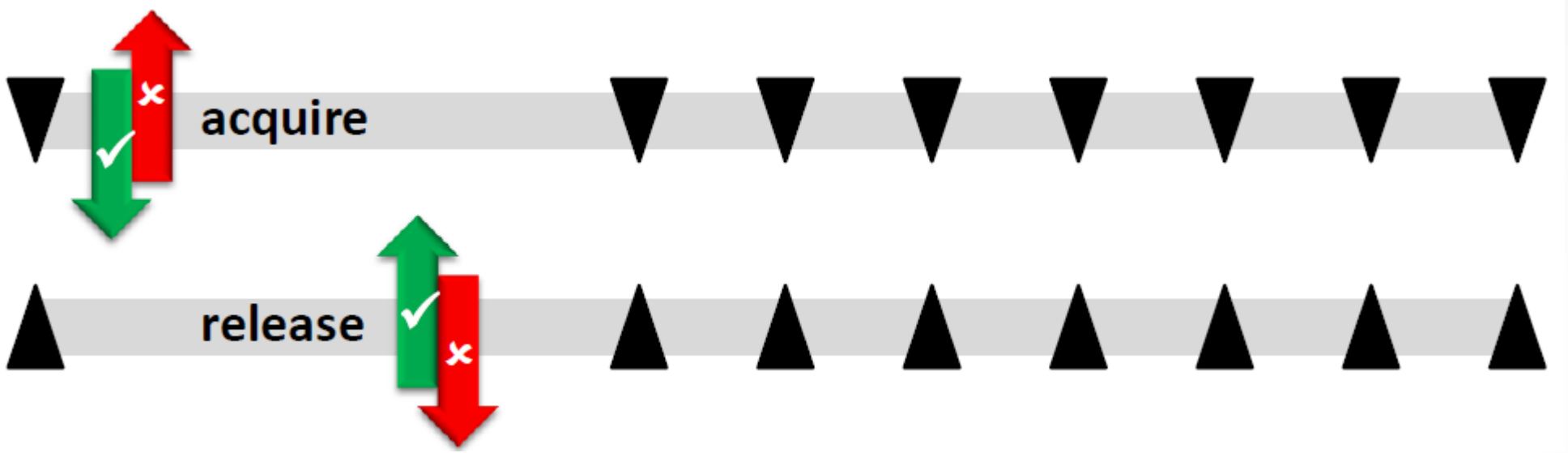
```
z = "everything";
mut.lock();

y = "universe";


mut.unlock();
x = "life";
```

You can safely move into a critical section
No reordering across *acquire* and *release (Acq/Rel)*
SC -> *Release* and *acquire* cannot be moved across each other

Source [5]: Herb Sutter slides

University of Vienna

ACQUIRE AND RELEASE (REMINDER)



"One-way barriers": An "acquire barrier" and a "release barrier."

- Note: These are fundamental hardware and software concepts.
- More precisely: A release store makes its prior accesses visible to a thread performing an acquire load that sees (pairs with) that store.

Source [5]: Herb Sutter slides

University of Vienna

C++ ATOMIC OPERATIONS

The CPU has a set of instructions that can perform some operations on some types atomically

- When such instruction is executed, the value in memory is changed in such a way that *no other thread can change it while the operation is ongoing*
- Executes as a single transaction
  - Primitive operation that does not suffer from data races
  - Can be used to implement higher-level concurrency mechanisms, such as locks, threads, and lock- free data structures
- Supported by hardware
  - **Atomic load, store, increment, add, exchange,...**
  - **compare exchange**
    - "if the existing value is X (expected), replace it with Y (desired), return the old value"
    - useful for building more advanced operations
  - memory_order_seq_cst by default (we will come to this)

C++ ATOMIC OPERATIONS

`atomic` class (`<atomic>`)

- A template class representing a value of the type with atomic operations

The operations are simple and straightforward

- You always get atomicity, but the synchronization part is configurable
- More expensive than normal operations
- In memory, not registers
- Needs to synchronize with other cores (and CPUs)
  - And their caches!
- It's the memory model where things get tricky!

```
atomic<int> counter = 0;
```

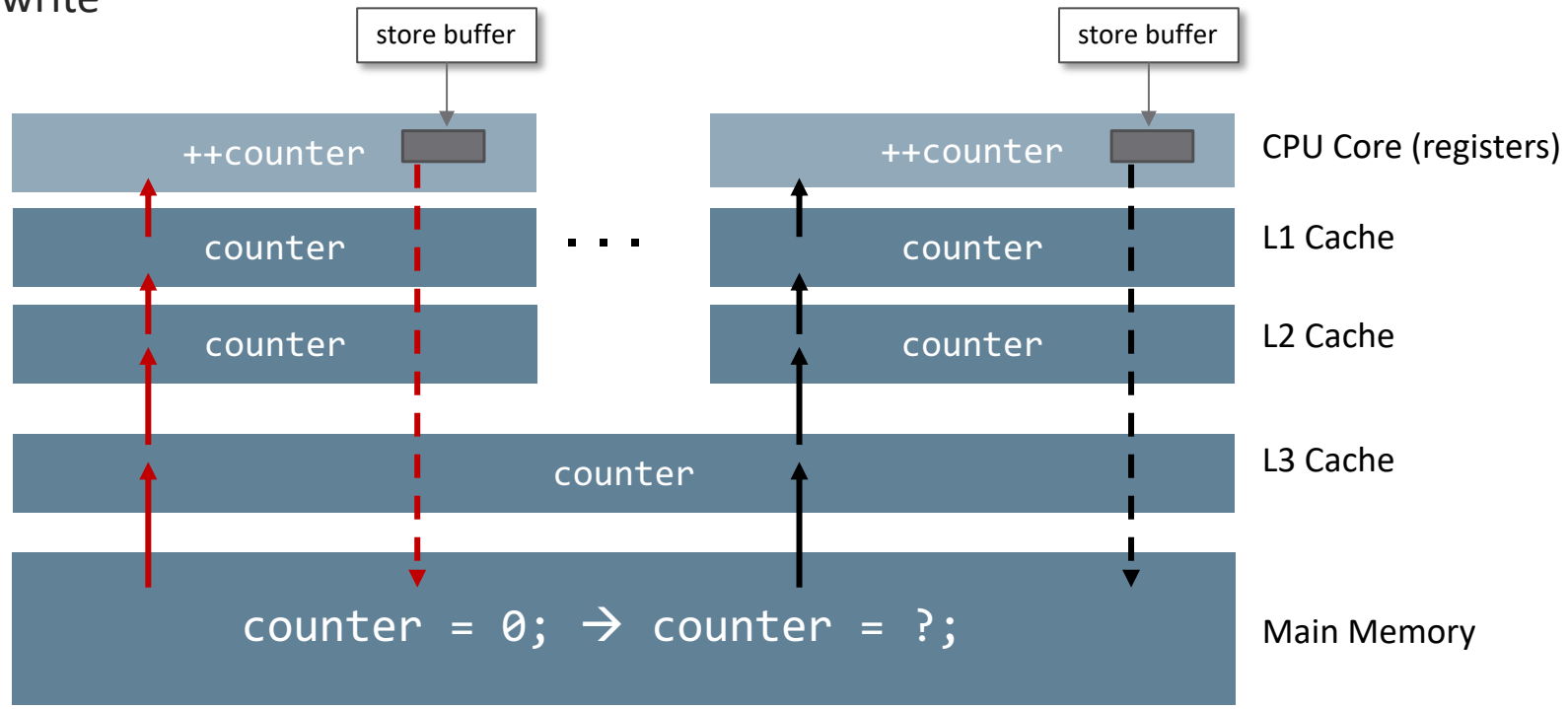
Thread 0	Thread 1
`++counter;`	`++counter;`

Possible results?

CACHES, MEMORY ACCESSES (REMINDER)

Simplified example of what may happen behind the scenes

- Read-modify-write



C++ ATOMICS :: SIMPLE EXAMPLE

```
atomic<int> counter = 0;

jthread th1([&]() {
   cout << ++counter;
});

jthread th2([&]() {
   cout << ++counter;
});

th1.join(); th2.join();
```

Thread 1

Thread 2

Possible results: 12,21

- Increment happens in order (0->1 and 1->2)
- But printing may take time

Impossible results: 11,22

Not equivalent to: `++counter;`

`cout << counter;`

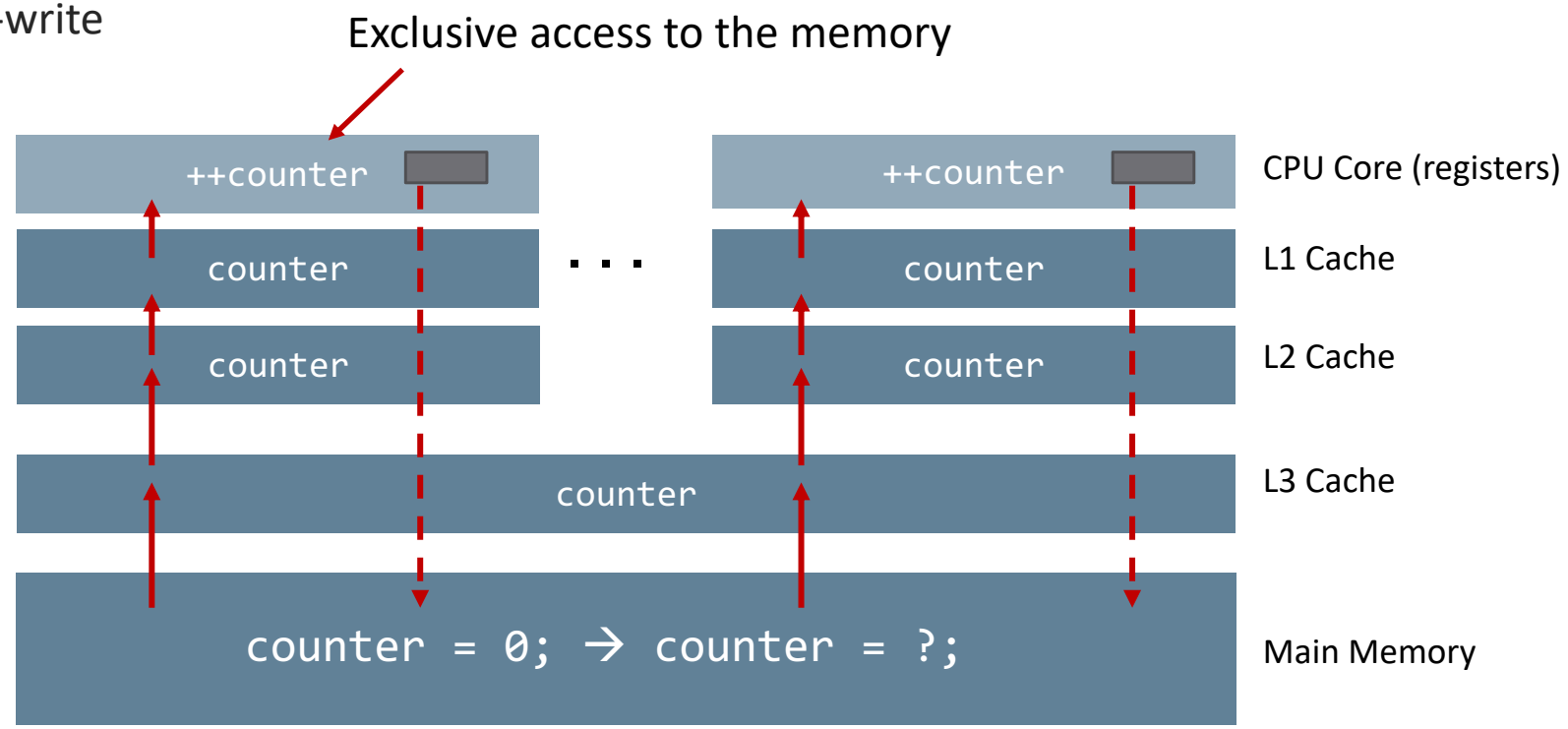
- another load

University of Vienna

CACHES, MEMORY ACCESSES :: EXAMPLE WITH ATOMIC

Simplified example of what may happen behind the scenes

- Read-modify-write




```
atomic<int> counter = 0;
```

Guaranteed not to see intermediate state if all operations are atomic

Source: [4]

CACHES, MEMORY ACCESSES :: OBSERVATIONS

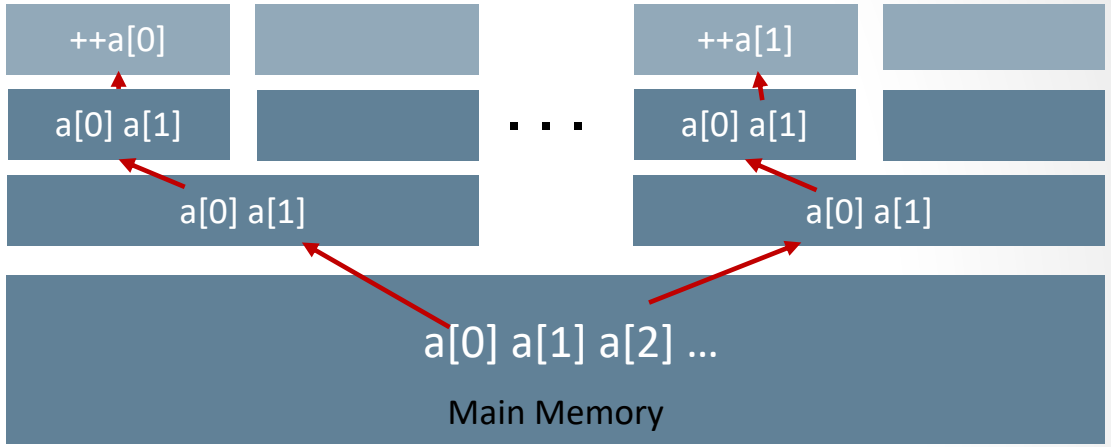
Cache lines sharing

- E.g., 64 bytes

Exclusive access to whole cache line

→ **False sharing**

- Different variables on the same cache line



Note: atomic write operations wait for cache line access!

- You still have no data races, but there may be significant performance penalties
- Be careful of accessing data on the same cache line
  - This may not always be obvious
  - Try to make thread access different cache lines when possible (e.g., introducing blocking in your loops)

Source [4]

University of Vienna

C++ ATOMIC OPERATIONS

`atomic` class (`<atomic>`)

- A template class representing a value of the type with atomic operations
- Specializations of the atomic template: `atomic`<int> i;
  - one integer with atomic operations
  - `load, store, is_lock_free, exchange, compare_exchange_weak, compare_exchange_strong, ++, --, +=, -=, &=, |=, ^=, fetch_add, fetch_sub, fetch_and, fetch_or, fetch_xor`

No atomic multiply! ☹

- Not copyable or moveable
- Designed for simple built-in types, for large objects may use locks
- Initialization of an atomic variable is not an atomic operation
- **What types can be atomic?**
  - **Basic types** usually integers, up to 8 bytes, pointers
  - You can use them for any *trivially copyable types*
    - Scalar types, contiguous chunks in memory (you can copy it with memcpy)
    - No virtual functions or virtual base classes, no user-defined copy operations
  - `is_lock_free()` member function
    - **true** if the atomic operations on the objects of this type are lock-free
      - otherwise, it may be implemented differently, e.g., using mutexes
    - Returns true on all major implementations for integral and pointer types

C++ ATOMIC OPERATIONS

Operations on atomic types are atomic (?)

- Assignments (reads and writes)
- Increment and decrement for raw pointers

int:

- ++, +=, -, -=, |=, &=, &^

bool:

- Valid, no special operations

double:

- Valid, no special operations
- No increment

However

- Expressions may not be atomic (be careful)
- ++x; x+=1; x=x+1; → not the same
- ++x vs x++ (fetch_add, add_fetch)

```
std::atomic<int> x(0);

++x;          // Pre-increment
x++;          // Post-increment
x +=1;        // Increment
x|=1;         // Bit set
x *=2;        // Does not compile
y = x+1;      // Read x atomically
x = y+1;      // Write x atomically

// different
x = x+1;      // Atomic read followed by atomic write
x = x*2;      // Atomic read followed by atomic write
```

Source [4]

C++ ATOMIC OPERATIONS :: EXPLICIT USE

`fetch_add (T arg, std::memory_order order)` ← We will get to this part later!

- Replaces the current value with the result of arithmetic addition of the value and arg
- For `int` types:
  - `x.fetch_add(y);`
    - non-atomic: `x+=y;`
  - `int z = x.fetch_add(y);`
    - non-atomic: `z=(x+=y)-y;`

Also `fetch_sub`, `fetch_add`, `fetch_or`, `fetch_xor`, ...

More verbose but less error prone

- Including load and store

Operator overloads provided only for atomic operations

- Incorrect code does not compile

Any expression with atomic variables will not be computed atomically

- Easy to make mistakes

Member functions may be less error prone (explicit)

Source [4]

C++ ATOMIC OPERATIONS :: EXPLICIT USE

Load, store

- Explicitly read and write

```
T y = x.load();        (T y = x;)
x.store(y);            (x = y;)
```

Exchange

```
T z = x.exchange(y);  (atomically z = x; x = y;)
```

Atomic Swap (read-modify-write)

Z	X	Y
0	1	2
1	2	2

Compare and Swap (CAS) **(we revisit this later)**

- Conditional exchange
- `bool success = x.compare_exchange_strong(expected, desired);`
  - If `(x==expected)`, make `x=desired` and return `true`, otherwise set `expected=x` and return `false` (through the first parameter)
- Used in many lock-free algorithms

University of Vienna

CRITICAL REGIONS (CAN WE DO IT WITH ATOMIC?)

A piece of code that mustn't be simultaneously accessed by more than one thread

- Must execute in isolation wrt. other program code

Locks (Mutexes)

enter critical region / lock acquire

```
my_mutex.lock();
... read/write x
my_mutex.unlock();
```

```
{ lock_guard<mutex> l(my_mutex);
    ... read/write x
}
```

exit critical region / lock release

Atomics

```
while( whose_turn != me ) {}
… read/write x
whose_turn = someone_else;
```

enter critical region / atomic read (acquire)

exit critical region / atomic write (release)

→ Let us first see what atomics are

source [5]

University of Vienna

MEMORY MODEL: SC, ACQUIRE/RELEASE, TOTAL ORDER

Transitivity/causality

- It must be impossible for the assertion to fail – wouldn't be SC

```
// Thread 1
g = 1;
x = 1;
```

```
// Thread 2
if (x == 1)
   y = 1;
```

```
// Thread 3
if (y == 1)
   assert(g == 1);
```

Total Store Order

- It must be impossible to print both messages – it would not be SC.

```
// Thread 1
x = 1;
```

```
// Thread 2
y = 1;
```

```
// Thread 3
if (x == 1 && y == 0 )
   print("x first");
```

```
// Thread 4
if (y == 1 && x == 0 )
   print("y first");
```

x and y are std::atomic, all variables initially zero

Source [5]: Herb Sutter slides

C++ ATOMIC OPS :: MEMORY ORDER

enum passed to atomic operations to specify how they should behave w.r.t. the memory model

- Limits reordering done by the compiler, may issue barrier (fence) instructions to CPU
- `memory_order_relaxed` → only ensures atomicity of the operation, faster because there are less overheads
- `memory_order_release`
  - Ensure that all preceding writes (to non-atomic, relaxed atomic) go to memory and make the changes available
- `memory_order_acquire` (the second half of release-acquire pair, on the same atomic variable)
  - Ensure that all subsequent reads see the changes made by the thread that did the release before the release
- `memory_order_acq_rel`
  - Both acquire and release (works on the same atomic variable)
- memory_order_seq_cst
  - Acquire (if contains load), release (if contains store), or both (for read-modify-write)
  - There is a global total order on all such operations
  - Default, simpler, but may not be needed and it is more expensive then acq_rel or anything above.
- memory_order_consume
  - Similar to acquire, forms release-consume pair
  - Only ensure visibility of changes that have dependency relationship with the release and consume operations
  - Not relevant for most architectures, no compiler actually does it anyway; lifted to acquire

MEMORY_ORDER_SEQ_CST

```
x=0, y=0;

thread A: x=1;

thread B: y=1;

thread C: while (x==0) {}; if (y==0) cout<<“thread A was first”;

thread D: while (y==0) {}; if (x==0) cout<<“thread B was first”;
```

Without total ordering:

- C sees the result of x=1 but not y=1
- D sees the result of y=1 but not x=1
- We can get both messages

With total ordering:

- The sequence for xy is either 00 → 10 → 11 or 00 → 01 → 11
- In the first case, after D has seen y==1, it has to see x==1
- In the second case, after C has seen x==1, it has to see y==1

Note: we may easily see no message

- Let’s assume 00 → 10 → 11
- Thread D can print no message, as x changes first
- Thread C sees that x==1, but before it gets a chance to read y, thread B runs and sets it to 1

RELAXED ORDER EXAMPLE

It is entirely architecture-specific whether a given memory order makes sense

- Can be more efficient, but hard to reason about since threads do not have to agree on order

```
// thread 1:
r1 = y.load(memory_order_relaxed);
x.store(r1,memory_order_relaxed);
```

```
// thread 2:
r2 = x.load(memory_order_relaxed);
y.store(42,memory_order_relaxed);
```

Allowed: r2 = 42

```
y.store(42,memory_order_relaxed);
r1 = y.load(memory_order_relaxed);
x.store(r1,memory_order_relaxed);
r2 = x.load(memory_order_relaxed);
```

Allowed ordering, why?
→ x and y are different variables, so no guaranteed ordering

Example: `counter.fetch_add(1, memory_order_relaxed)`?

Source [2]

RELAXED ORDER EXAMPLE

Some patterns can benefit from relaxed order

- e.g., simple incrementing a counter
- Typically, implemeted as a type that encapsulates the desired semantics and hides the complexity

```
// thread 1:
while () {
   ...
   if ( ... )
      ++counter;
   ...
}
```

```
// thread 2:
...
start_threads(); // sync point
...
join_threads(); // sync point
...
cout << counter << endl;
```

University of Vienna

RELAXED ORDER EXAMPLE

Some patterns can benefit from relaxed order

- e.g., simple incrementing a counter
- Typically, implemeted as a type that encapsulates the desired semantics and hides the complexity

```
// thread 1:
while () {
   ...
   if ( ... )
      counter.fetch_add(1, memory_order_relaxed);
   ...
}
```

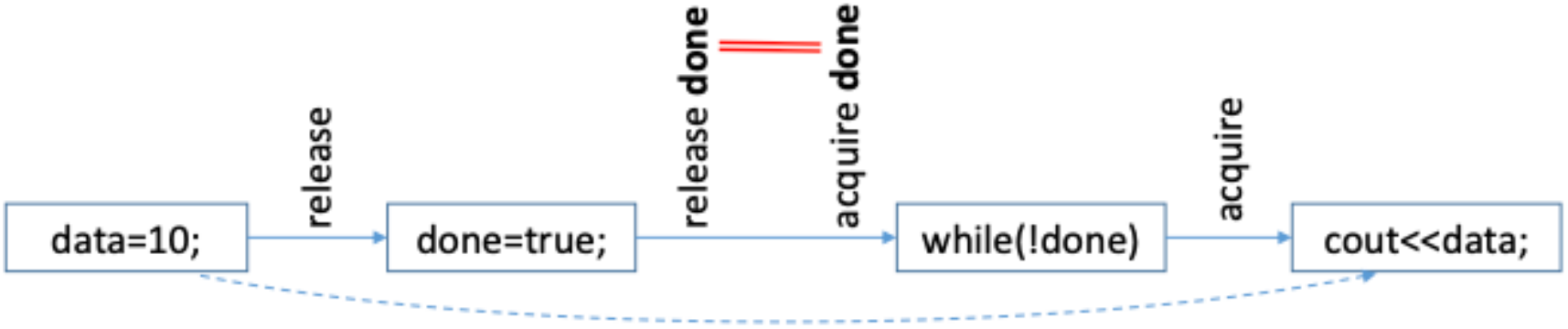
```
// thread 2:
...
start_threads(); // sync point
...
join_threads(); // sync point
...
cout <<
counter.load(memory_order_relaxed)
<< endl;
```

C++ ATOMIC :: MEMORY ORDER ANTI-EXAMPLE

```
int data; atomic<bool> done = false;
```

```
// thread A

data=42;
done.store(true, memory_order_relaxed); // problem!
```

```
// thread B

while(!done.load(memory_order_relaxed)) this_thread::yield();
cout << data << endl;
```

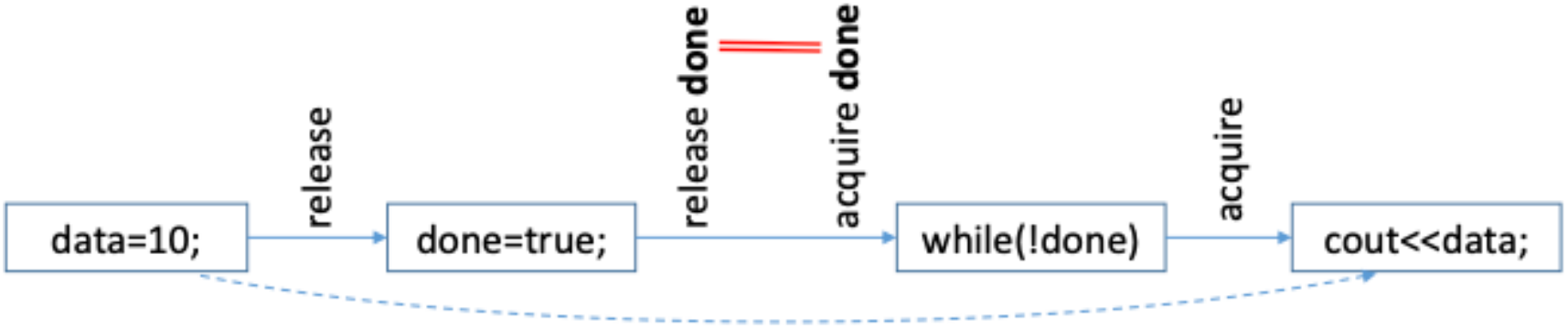


C++ ATOMIC :: MEMORY ORDER EXAMPLE

```
int data; atomic<bool> done = false;
```

```
// thread A

data=42;
done.store(true, memory_order_release);
```

```
// thread B

while(!done.load(memory_order_acquire)) this_thread::yield();
cout << data << endl;
```



C++ ATOMIC :: MEMORY ORDER ANTI-EXAMPLE

```
int data; atomic<int> a = 0; atomic<int> b = 0;
```

```
// thread A

data=10;
a.store(1, memory_order_release);
```

```
// thread B

while(!b.load(memory_order_acquire)) this_thread::yield();
cout << data << endl;
```

There is no relation between store of a in A and acquire of b in B, so no synchronization is established

Will only work if someone reads the 1 from a with acquire and writes b=1 with release:

```
// thread C

while(!a.load(memory_order_acquire)) this_thread::yield();
b.store(1, memory_order_release);
```

C++ ATOMIC :: MEMORY ORDER ANTI-EXAMPLE

```
int data; atomic<bool> done = false;
```

```
// thread A

data=10;
done.store(true, memory_order_release);
```

```
// thread B

while(!done.load(memory_order_acquire)) this_thread::yield();
cout << data << endl;
```

```
// thread C

data=20;
done.store(true, memory_order_release);
```

Do we get 10 or 20 in B? The acquire in B reads true value written in either A or C. This establishes synchronization with either A or C, but not both.

C++ ATOMIC :: MEMORY ORDER ANTI-EXAMPLE

```
int data; atomic<int> state = false;
```

```
// thread A

data=1;
state.store(1, memory_order_release);
```

```
// thread B

while (state.load(memory_order_acquire) != 1) this_thread::yield();
data *= 2;
state.store(2, memory_order_release);
```

```
// thread C

while (state.load(memory_order_acquire) != 2) this_thread::yield();
cout << data << endl;
```

This works, because we know that B acquires state after A released it, but also C acquires state only after it has been released by B (otherwise it would not see state==2). Therefore we have *release-acquire* from A to B and then *release-acquire* from B to C. The statement data*=2 acquires value released by A and then releases the new value to be acquired by C.

C++ MEMORY MODEL :: HAPPENS-BEFORE

A relation between statements in C++ based on the program and synchronization

- Example (*sequenced-before*): { A; B; }
  - Provides **intra-thread** ordering
- Example (*synchronizes-with*): release(x)-->acquire(x)
  - Provides **inter-thread** ordering
- Transitivity: { A; release(x); } --> { acquire(x); B; }
- Synchronization does not only mean atomics → also mutexes, thread creation, join, ...

If A happens-before B, then B is guaranteed to see the value written by A

- Unless it is also modified by C, (A *happens-before* C) and (C *happens-before* B)
- Between memory operations in different threads

It's possible that neither (A *happens-before* B) nor (B *happens-before* A)

- A and B are not synchronized
- If A writes and B reads the same variable non-atomically, you are probably in trouble - data race

C++ MEMORY MODEL: SUMMARY

The memory model, as specified in the ISO C++ standard, represents a contract between the implementers and the programmers to ensure that most programmers do not have to think about the details of modern computer hardware.

"What can programmer assume about what thread sees when it looks at a value from memory" [2]

Different Memory Orderings are supported

- Default → memory_order_seq_cst
- Can be relaxed to (potentially) improve performance (e.g., acquire-release pair around a single variable)

An atomic operation ensures that the state of the memory is as required by the specified memory order

- Can be concurrently updated without introducing a data race, no half updates visible, lock-free and supported by hardware when possible
- Often they are used in combination with non-atomic variables (as they are more expensive than normal ops)
  - This introduces complexities and needs to be handled correctly
  - Basis for many lock-free algorithms

SYNCHRONIZATION IN C++: ATOMICS VS MUTEXES

Atomics, Locks, higher-level mechanisms

Updating shared state

- Atomics just make the change / mutex may block a thread if there is a collision
- Atomic update only of a one simple variable / mutex may protect shared state of any complexity
- Atomics are in user space / mutexes may need to go to kernel space
- Managing non-trivial shared state with atomics can be complex
  - Have you really considered **all** possibilities?
- Mutexes let you freeze the world and make all the changes that you need, and read a fully consistent state
- Making changes visible to other threads → finer control with atomics

Waiting

- Atomics have a low overhead
- Mutexes (and futures, condition variables,…) can suspend the thread
- Atomics for rarely contested locks
- Synchronization should be minimized as it may have a serious impact on the performance

COMPARE-EXCHANGE

Lock-free algorithms (not wait-free)

```
std::atomic<int> x{0};
int x0 = x;
while (!x.compare_exchange_strong(x0, x0+1)) {}
```

Weak and strong versions

- Two value arguments: `T& expected, T desired`
- `x.compare_exchange_strong(expected, desired);`
  - `if (x == expected) { x = desired; return true; }`
  - `else {expected = x; return false; }`
- `x.compare_exchange_weak(expected, desired);`
  - Can spuriously fail and return false even if `x == expected`
    - `e.g., double check locking pattern`
  - Because of the hardware support

	X	E	D	success
CAS	1	2	3	
result	1	1	3	false
CAS	1	1	3	
result	3	1	3	true

"Am I the one who get to change the value from expected to desired"

EXAMPLE :: COMPARE-EXCHANGE WITH LINKED-LIST

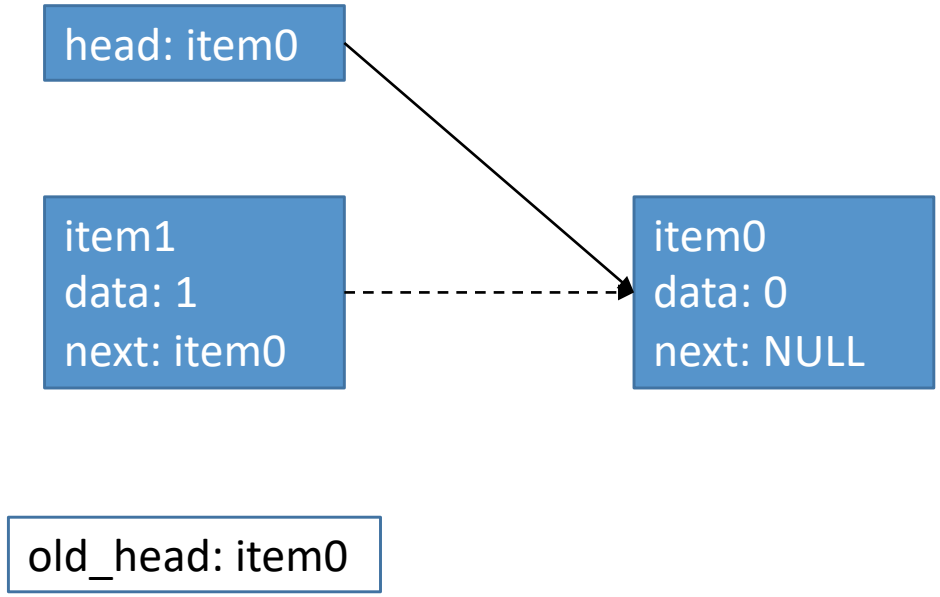
```
atomic<item *> head = 0;
vector<thread> workers;
atomic<int> counter = 0;
int num_workers = thread::hardware_concurrency();
for (int i = 0; i < num_workers; ++i)
{
   workers.push_back(thread([&, i]()
   {
      for (int j = 0; j < 10; ++j)
      {
         item *new_item = new item(i * 10 + j);
         item *old_head = head.load();
         new_item->next = old_head;
         while (!head.compare_exchange_weak(old_head, new_item)) {
            //the old_head may have been updated by the
            //compare_exchange, update new_item->next
            new_item->next = old_head;
         }
      }
   }));
}
for (int i = 0; i < num_workers; ++i)
{
   workers[i].join();
}
```

atomic pointer! (points to non-atomic memory)

head has not changed

new head

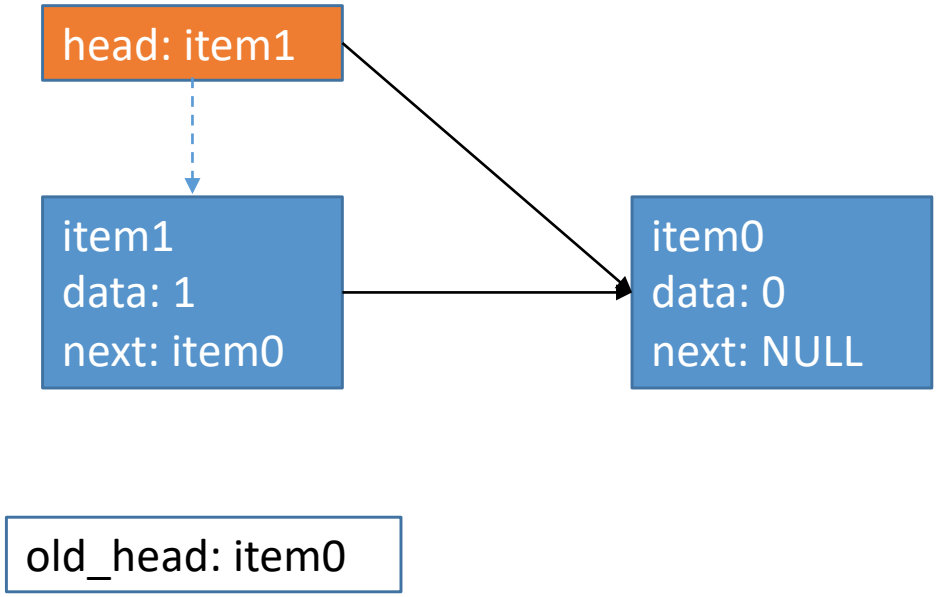
EXAMPLE :: COMPARE-EXCHANGE WITH LINKED-LIST




```
item* new_item = new item(i * 10 + j);
item* old_head = head.load();
new_item->next = old_head;
while (!head.compare_exchange_weak(old_head, new_item))
```

conditional swap — expected — desired

EXAMPLE :: COMPARE-EXCHANGE WITH LINKED-LIST

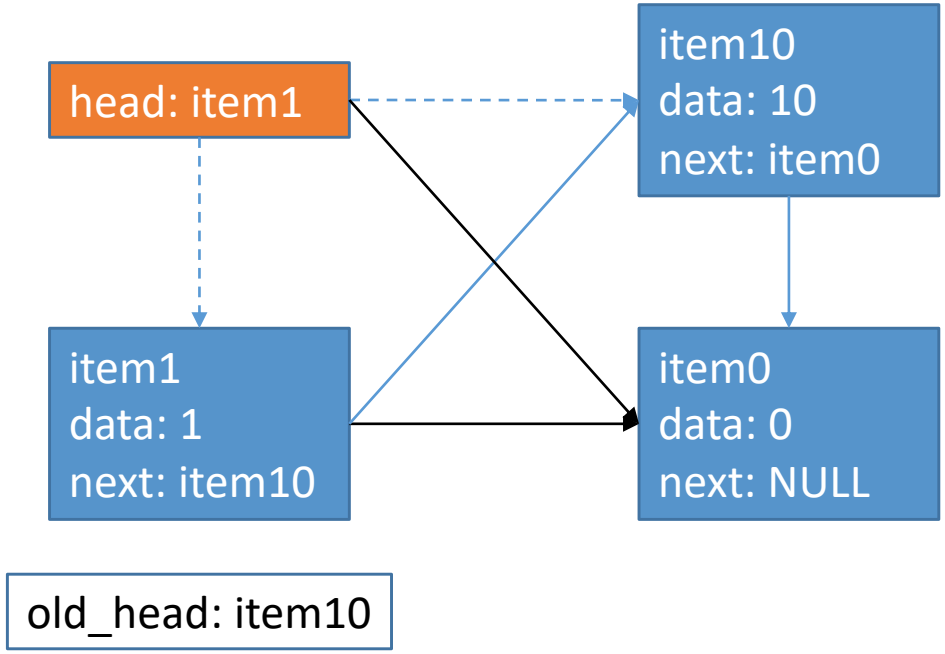



```
item* new_item = new item(i * 10 + j);
item* old_head = head.load();
new_item->next = old_head;
while (!head.compare_exchange_weak(old_head, new_item))
```

conditional swap — expected — desired

University of Vienna

EXAMPLE :: COMPARE-EXCHANGE WITH LINKED-LIST




```
item* new_item = new item(i * 10 + j);
item* old_head = head.load();
new_item->next = old_head;
while (!head.compare_exchange_weak(old_head, new_item)) {
   new_item->next = old_head;
}
```

For performance you should use different memory orders, but it should be at least ***release*** to make data visible

EXAMPLE :: SPIN LOCK

```
struct spin_mutex
{
   void lock() {
      bool state = false;
      while (!locked.compare_exchange_weak(state, true, memory_order_acquire))
      {
         // the lock was locked, so the state was changed to true
         state = false;
      }
   }

  void unlock() {
    locked.store(false, memory_order_release);
  }

  private:
    atomic<bool> locked = false;
};
```

OTHER ATOMICS

`atomic_flag`

- Flag, one bit atomic test and set
- Guaranteed lock-free

`shared_ptr<T>`

- atomic operation on shared pointers

`atomic_thread_fence` function

- Explicit memory fence
- Used in combination with another atomic operation
- Thread A:
  - `fence(release);`
  - `atomic_w(v1,relaxed); atomic_w(v2,relaxed); atomic_w(v3,relaxed);`

Thread B:

  - `atomic_r(v1,relaxed); atomic_r(v2,relaxed); atomic_r(v3,relaxed);`
  - `fence(acquire);`
- Ensures thread A finishes all writes before any of the atomic writes and thread B starts all reads after the atomics are read

SPIN LOCK :: ATOMIC FLAG

```
struct spin_mutex
{
   void lock() {
      while (locked.test_and_set(std::memory_order_acquire)) ; // NOP
            // test_and_set sets the flag to true;
   }

   void unlock() {
      locked.clear(std::memory_order_release); // set to false
   }
private:
   atomic_flag locked = ATOMIC_FLAG_INIT; // set to false
};

spin_mutex sm;
void worker() {
   sm.lock();
   // critical section
   sm.unlock();
}
```

*Note: we are actively waiting!

More Examples

DYNAMIC WORK ALLOCATION WITH ATOMIC

```
#include <iostream>
#include <vector>
#include <thread>
#include <atomic>

void worker(std::atomic<int>& next_index, std::vector<int>& items) {
  while (true) {
    int index = next_index.fetch_add(1);

    if (index >= items.size()) {
      return;
    }
    process(items[index]);
  }
}


int main() {
  std::vector<int> items(10, 0);
  std::atomic<int> next_index = 0;

  std::jthread t1(worker, std::ref(next_index), std::ref(items));
  std::jthread t2(worker, std::ref(next_index), std::ref(items));

  t1.join();
  t2.join();

  for (int x : items) {
    std::cout << x << " ";
  }
  std::cout << "\n";
}
```

We need sync here?

```
void process(int& item) {
  item++; // simple work
}
```

Dynamic work allocation:
→ Assign work to compute units in response to their progress

DYNAMIC WORK ALLOCATION WITH ATOMIC

```
struct worker
{
void process(int& i) { ++i; this_thread::sleep_for(10ms); }
void operator()(atomic<int>& counter, vector<int>& items)
{
   for (;;)
   {
      int index = counter++;
      if (index >= items.size()) return;
      process(items[index]);
   } }
};
vector<int> items_to_process(32, 0); vector<thread> workers;
atomic<int> counter = 0;
int num_workers = thread::hardware_concurrency();
for (int i = 0; i < num_workers; ++i)
{
   workers.push_back(jthread(worker(), ref(counter), ref(items_to_process)));
}
for (int i = 0; i < num_workers; ++i)
{
   workers[i].join();
}
```

Exclusive slot, no two threads have the same value
We could also do counter.fetch_add(1);

*We are adding an element to a vector here, since counter is atomic, each thread will have a different index value
We uses atomic index to a non-atomic memory → basis for lock-free algorithms

:: THE LAST SLIDE

Questions?

SOURCES, REFERENCES, MATERIALS

1. **C++ Concurrency in Action, Anthony Williams**
2. **The C++ Programming Language, 4th Edition, Bjarne Stroustrup**
3. **Slides Dr. Jiri Dokulil**
4. **Fedor Pikus Slides “C++ atomics, from basic to advanced. What do they really do?”**
5. **Herb Sutter Slides “atomic<> Weapons”**
6. **Parallel and High Performance Computing, Robert Robey, Yuliana Zamora**
7. **https://en.cppreference.com,**